Licitház fejlesztői dokumentáció

Készitette: Szokolay Márk, Csermák Viktor, Deákvári Marcell

1. Cél és áttekintés

Ez a dokumentum a Licitház Laravel projekt helyi fejlesztéséhez, futtatásához, megértéséhez és bővítéséhez ad gyakorlati útmutatót. A célja, hogy egy új fejlesztő a repository megnyitásától el tudjon jutni a lokális futtatásig, majd onnan a fő üzleti folyamatok és kódszerkezet megértése.

A projekt egy aukciós rendszer, ahol:

- a látogatók böngészhetnek a termékek között,
- a regisztrált felhasználók licitálhatnak,
- az aukció nyertese fizethet,
- az admin felhasználók kezelhetik a termékeket, felhasználókat és fizetéseket.

2. Használt technológiák

Backend:

- PHP 8.2
- Laravel 12
- Eloquent ORM
- Laravel Sanctum
- Laravel UI
- Laravel API

Frontend:

- Blade sablonok
- Vite
- Tailwind CSS

Fejlesztői eszközök

- Composer
- npm
- opcionálisan XAMPP vagy más helyi PHP + MySQL környezet

3. Előfeltételek

Lokális fejlesztéshez az alábbiak legyenek telepítve:

- PHP 8.2 vagy újabb
- Composer

- Node.js (lehetőleg a legujabb) és npm
- MySQL vagy MariaDB
- Git

Windows alatt a projekt jól illeszkedik XAMPP alapú fejlesztői környezetbe. A jelenlegi workspace is ilyen struktúrában fut. Figyelem MacOs és Linux platformon máshogy müködhet a program előkészülete.

4. Projektstruktúra röviden

A legfontosabb mappák:

- app/Http/Controllers: webes végpontok és üzleti folyamatok belépési pontjai
- app/Models: Eloquent modellek (Product, Bid, Payment, User)
- app/Policies: jogosultsági szabályok
- database/migrations: adatbázis séma
- database/seeders: fejlesztői tesztadatok
- resources/views: Blade frontend oldalak
- resources/css és resources/js: frontend entry pointok Vite-hez
- routes/web.php: web route-ok
- tests: unit és feature tesztek

5. Első telepítés

Nyisd meg a projektet a Licithaz mappában, majd futtasd a következőket:

-composer install
-copy .env.example .env
-php artisan key:generate

6. Környezeti változók

Az alap .env.example jelenleg MySQL fejlesztői környezetre van előkészítve:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=licithaz
DB_USERNAME=root
DB_PASSWORD=
```

Ezen felül érdemes figyelni az email beállítására. Fontos ha saját email-t használsz akkor **ne a saját jelszódat** hanem a gmail beállításokban lehet generálni egy jelszót hozzá amit ne osszon meg senkivel!

MAIL_MAILER=smtp
MAIL_SCHEME=smtp
MAIL_HOST=smtp.gmail.com
MAIL_PORT=587
MAIL_USERNAME=sajat teszt email
MAIL_PASSWORD="a gmail legeneralja a beallitasokban"
MAIL_FROM_ADDRESS="sajat teszt email"
MAIL_FROM_NAME="Licithaz (mi legyen a kuldo cime)"

Ha nem a default MySQL kapcsolatot használod, a dokumentáció további részei előtt mindig ellenőrizd a .env értékeket.

7. Adatbázis inicializálás, adatbázis létrehozása

Hozd létre a licithaz adatbázist MySQL-ben.

7.2 Migrációk futtatása

php artisan migrate

7.3 Seedelés

php artisan db:seed

A seedelés jelenleg létrehoz több alap felhasználót és termékadatokat. A DatabaseSeeder ezeket hozza létre:

- admin user: admin@example.com
- teszt user: test@example.com
- tiltási teszthez user: ban@example.com

Az összes felhasználó seedelt jelszava: **password**.

7.4 Storage link

Termékképekhez a public storage link szükséges:
    -php artisan storage:link

Ez különösen fontos, mert a termékképek mentése a storage/products útvonalra történik, és a storage/… lehet elérni.

8. Frontend telepítés

Telepítsd a frontend függőségeket:
-npm install

Fejlesztői mód elindítása:
-npm run dev

A Vite entry pointok:
- resources/css/app.css
- resources/js/app.js

9. Lokális futtatás

9.1 Egyszerű indítás

Backend:
-php artisan serve

Frontend:
-npm run dev

Ezeket külön párhuzamosan futó konzolokon kell futattni.

9.2 Ajánlott fejlesztői indítás

A projekt tartalmaz egy kényelmes scriptet:

composer dev

Ez egyszerre indítja el:

- a Laravel szervert,
- a queue listenert,
- a Vite dev szervert.

Mivel a queue kapcsolat adatbázis alapú, a háttérfolyamatok és bizonyos aszinkron feldolgozások esetén ez a mód közelebb áll a tényleges fejlesztői működéshez.

10. Fontos napi parancsok

```
php artisan migrate
php artisan migrate:fresh --seed
php artisan db:seed
php artisan optimize:clear
php artisan route:list
php artisan test
```

11. Fő üzleti modulok

11.1 Termékek

A ProductController kezeli:

- terméklista,
- termék részletező,
- keresés,
- termék létrehozás (admin),
- termék szerkesztése (admin),
- soft delete (admin),
- restore (admin),
- force delete (admin).

A publikus lista jelenleg nem mutatja a sold státuszú elemeket ha kifizették de a user megtudja tekinteni a fizetésnél őket és az admin is az admin felületen.

11.2 Licitek

A BidController felel a licitek rögzítéséért.

Licit rögzítés előtt a rendszer ellenőrzi, hogy:

- a licitálás nyitva van-e időben,
- a termék státusza active-e,
- a kérés validált értéket tartalmaz-e,
- a felhasználó jogosult-e licitet létrehozni.

Új legmagasabb licit esetén az előző vezető licitáló e-mail értesítést kap.

11.3 Aukció lezárás

A ProductController show metodusa hívja az AuctionResolutionController logikáját, amely lezárja időben a lejárt aukciót.

11.4 Fizetés

A PaymentController kezeli:

- checkout oldal megjelenítése,
- fizetés rögzítése,
- saját fizetések listázása,
- admin összes fizetés listája.

Fizetni csak az aukció nyertese tud, és csak akkor, ha a termék státusza pending_payment másnéven lejárt az aukciós idő.

Sikeres fizetéskor:

- a termék státusza sold lesz,
- létrejön egy payments rekord,
- a nyertes visszaigazoló e-mailt kap és kifizetheti a terméket.

11.5 Profil és felhasználókezelés

Bejelentkezett felhasználó:

- megtekintheti a profilját,
- licitálhat
- frissítheti a profilját (jelszó, profilkép),

Admin:

- eléri a dashboardot,
- szerkeszthet felhasználókat ha szükséges,
- tilthat felhasználót,
- megtekintheti a fizetéseket.

12. Jogosultságok

A projekt policy alapon védett.

Fő szabályok:

- ProductPolicy: terméket létrehozni, módosítani, törölni, visszaállítani és végleg törölni csak admin tud
- UserPolicy: user lista, tiltás és szerkesztés admin joghoz kötött
- fizetési admin nézet policy ellenőrzéssel védett
- licit létrehozás auth és BidPolicy ellenőrzés mögött fut

Biztonsági konvenciók, amelyeket fejlesztés közben tarts meg:

- a termékmódosító route-ok maradjanak auth és policy védelem mögött,
- publikus csak az index és show jellegű végpont legyen,
- admin dashboard és ban funkciók mindig kapjanak policy ellenőrzést,
- Blade view nevek kis- és nagybetűzését kezeld következetesen, mert Linux szerveren érzékeny lehet.

13. Route térkép röviden

A routes/web.php alapján a fő route csoportok:

- / kezdőoldal
- auth route-ok Auth::routes() alól
- /profile profilkezelés
- /products publikus terméklista és részletező
- /products/{product}/bids licit rögzítése

- /products/create, /products/{product}/edit és kapcsolódó admin termék route-ok
- /products/{product}/checkout és /products/{product}/pay fizetés
- /payments saját fizetések
- /dashboard admin dashboard
- /admin/payments összes fizetés admin nézetben
- /users/{user}/toggle-ban tiltás kapcsolása
- /admin/auctions/resolve admin aukció lezárás

14. E-mail és háttérfolyamatok

Az alkalmazás több helyen küld e-mailt:

- túlicitálás esetén,
- aukció megnyerésekor,
- fizetés visszaigazolásakor.

Fontos az env email paraméterek helyes beállítása amennyiben élő környezetben szeretnéd tesztelni. (Elején van leírás hogy kell).

15. Tesztelés

Alap tesztfuttatás:

php artisan test vagy composer test

16. Tipikus hibák és gyors ellenőrzések

16.1 Migrációnál elhasal

Ok: a MySQL adatbázis vagy a .env még nincs jól beállítva.

Teendő:

- hozd létre a licithaz adatbázist,
- ellenőrizd a DB host, port, user, password értékeket,
- futtasd külön a php artisan migrate parancsot.

16.2 Képek nem jelennek meg

Ok: hiányzik a storage link.

Teendő:

-php artisan storage:link

16.3 E-mailek nem látszanak

Ok: log mailer van használatban.

Teendő:

- ellenőrizd a storage/logs/laravel.log fájlt,
- vagy állíts be fejlesztői SMTP szolgáltatást.

16.4 Jogosultsági hiba admin oldalon

Ok: a seedelt user nem admin, vagy nincs belépve.

Teendő:

- jelentkezz be az admin@example.com fiókkal,
- ellenőrizd az is_admin mezőt az adatbázisban.

30. Laravel controller metódusok részletes magyarázata

Ez a fejezet kifejezetten arra szolgál, hogy a controller metódusok viselkedése fejlesztői szemmel is gyorsan érthető legyen.

30.1 Web ProductController

index():

- csak a nem sold státuszú termékeket listázza,
- lapozással paginate() adja át a nézetnek,
- célja a publikus terméklista kiszolgálása.

create():

- csak a létrehozó nézetet adja vissza,
- jogosultságot nem itt, hanem route/policy szinten kap.

store(StoreProductRequest $request):

- validált adatot vesz át,
- képet külön tárolja storeProductImage(...) segítséggel,
- mentés után redirectel.

show(int $id, AuctionResolutionController $auctionResolutionController):

- betölti a terméket a licitekkel és a licitáló userrel együtt,
- oldalletöltéskor meghívja az aukció lezárási logikát (resolveIfEnded),
- a nézetnek külön átadja az aktuális legmagasabb és minimum következő licitet.

edit(Product $product):

- szerkesztő nézetet ad vissza egy konkrét termékhez,
- route modell bindinggal kapja meg az adott termék szerkeszthető adatait.

update(UpdateProductRequest $request, Product $product, AuctionResolutionController $auctionResolutionController):

- validált mezőket frissít,
- ha új kép jött, törli a régit és menti az újat,
- dátummódosítás után újraértékeli az aukció státuszt,
- sikeres mentés után redirect + visszajelzés.

destroy(Product $product):

- törlés előtt eltávolítja a tárolt képet,
- soft delete-ot hajt végre,

restore(int $id):

- soft-deletelt terméket állít vissza,
- policy authorize hívást használ,
- admin restore flow célmetódusa.

forceDelete(int $id):

- végleges törlést hajt végre soft-deletelt terméken,
- kapcsolódó liciteket is törli,
- képfájlt is töröl public storage-ról,
- admin-only művelet.

search(Request $request):

- prefix alapú névkeresést csinál,
- üres keresésnél üres listát ad,
- maximum 5 találatot ad vissza JSON-ben.

30.2 BidController

store(StoreBidRequest $request, Product $product):

- ellenőrzi, hogy az aukció nyitott-e és a termék aktív-e,
- lekéri az előző legmagasabb licitet,
- létrehozza az új licitet a bejelentkezett userhez,
- szükség esetén outbid értesítést küld,
- visszairányít a termék oldalára státusz üzenettel.

30.3 PaymentController

checkout(Product $product):

- csak nyertes usernek engedi a fizetési oldalt,
- lekéri a nyertes licitet és annak összegét,
- visszaadja a payment nézetet.

pay(PaymentRequest $request, Product $product):

- újra ellenőrzi, hogy tényleg nyertes fizet,

- a terméket sold státuszra állítja,
- payment rekordot ment,
- visszaigazoló e-mailt küld,
- státusz üzenettel visszairányít.

index():

- a bejelentkezett felhasználó saját fizetéseit listázza,

adminIndex():

- policy ellenőrzés után listázza az összes fizetést,
- user + product kapcsolatot mutatva jeleníti meg.

ensureWinnerCanPay(Product $product):

- belső guard metódus,
- 403-mal megállít, ha nem a nyertes fizetne,
- 403-mal megállít, ha az aukció még nincs pending_payment állapotban (még nem zárult le).

winningBidOrAbort(Product $product):

- visszaadja a nyertes licitet,
- 409-et dob, ha nincs még nyertes licit.

30.4 Web ProfileController és UserController

ProfileController metódusok szerepe:

- profil megjelenítés,
- profil adatok frissítése,
- jelszócsere.

UserController metódusok szerepe:

- admin user szerkesztés,
- tiltás váltása (toggleBan) policy védelemmel.

31. API controller metódusok részletes magyarázata

31.1 Api\\AdminAuthController

login(Request $request):

- validálja az email/jelszó mezőket,
- ellenőrzi a hitelesítést,
- ellenőrzi az admin jogosultságot,
- Sanctum personal access tokent ad vissza.

logout(Request $request):

- törli az aktuális access tokent,
- ezzel az adott kliens session API-szinten megszűnik**.**

31.2 Többi Api controller

A Product CRUD műveleteket hajt végre hasonlóan mint a sima controllerben. A Payment, Bid, controller csak kilistáz (index()) funkció. A user controller is ugyanaz ugyanúgy listáz és tartalmazza a Ban/Unbant-t. Ezeket a válaszokat JSON formátumba küldi el a MAUI-nak.

32. Nem-controller, de fontos metódusok

32.1 Product modell metódusok

currentHighestBidAmount():

- visszaadja a legmagasabb licitet,
- ha még nincs licit, a starter bidet.

minimumNextBidAmount():

- kiszámolja a minimálisan adható következő licitet,
- jelenlegi szabály szerint +1000 Ft lépcsővel.

isBiddingOpen():

- időablak alapján eldönti, hogy nyitott-e az aukció.

winningBid():

- legmagasabb összeget adó licitet adja vissza.

isAuctionEnded():

- egyszerű időalapú lezárás-check.

userOutBid(Bid $newBid, ?Bid $oldHighestBid):

- ha valódi túlicitálás történt, e-mailt küld az előző vezető licitálónak.

notifyWinner(Bid $winningBid):

- aukció végi nyertes értesítést küld,
- fizetési linket is beágyaz az üzenetbe.

17. Release előtti ellenőrzőlista

- migrációk lefutnak tiszta adatbázison is,
- php artisan test sikeres,
- frontend build sikeres,
- storage link létezik,
- admin és normál user jogosultságok ellenőrizve vannak,
- payment és aukciós státuszváltások kézzel kipróbálva vannak,
- .env nincs véletlenül commitolva.

18. Ajánlott további dokumentáció

Ha a projekt tovább nő, érdemes külön dokumentumba kitenni:

- adatbázis diagram
- admin dashboard használati leírás,
- API vagy mobil integrációs pontok,

19. .NET MAUI admin alkalmazás áttekintése

A workspace nem csak a Laravel webalkalmazást tartalmazza, hanem egy külön .NET MAUI admin klienst is a licitAdminDashboard mappában. Ez a kliens közvetlenül a Laravel API-ra csatlakozik, és adminisztrációs feladatokat végez desktop felületen.

A MAUI app fő céljai:

- admin bejelentkezés tokennel,
- termékek listázása,
- termék létrehozása,
- termék szerkesztése,
- termék törlése,

- felhasználók listázása,
- tiltás és tiltás feloldása,
- licitek és fizetések listázása,
- egy adott felhasználó licitjeinek megtekintése.

21. MAUI fejlesztői setup

21.1 Előfeltételek

A MAUI alkalmazás futtatásához tipikusan ezek kellenek:

- Visual Studio 2022
- .NET 10 SDK vagy a projekthez illeszkedő preview/stabil SDK

21.2 Build és futtatás

Tipikus fejlesztői lépések:

1. Előbb induljon a Laravel backend és az API.
2. Ellenőrizd, hogy a MAUI app és a Laravel app ugyanazt a hostot használja.
3. Nyisd meg a licitAdminDashboard.slnx solutiont Visual Studio-ban.
4. Válaszd ki a targetet: Windows.
5. Buildeld és indítsd el az alkalmazást.

Fejlesztői szempontból kritikus, hogy a Laravel API ténylegesen elérhető legyen a MAUI appból. A jelenlegi kódban a host hardcode-olt.

21.3 Host és base URL

A MAUI app jelenlegi API alap URL-je:

- ApiService: http://localhost:8000/api/
- MainPage: http://127.0.0.1:8000/api/
- LoginPage: http://127.0.0.1:8000/api/admin/login

Ez lokális Windows fejlesztésnél működik, de Android emulátoron nem fog valószinüleg. Amennyiben Mobil alkalmazásként is akarja fejleszteni akkor ezt a

hardcoded címet meg kell változtatni mivel az emulátor általában ugyanazon a lokális címen van.

22. MAUI alkalmazás indulása és navigációja

22.1 App.xaml.cs

Az alkalmazás belépési pontja az App osztály. A CreateWindow metódus szerepe:

- létrehoz egy új Window példányt,
- a kezdőoldalnak egy NavigationPage(new LoginPage()) értéket ad,
- vagyis a kliens login képernyőről indul.

23. MAUI képernyők és felelősségeik

23.1 LoginPage

Feladata az admin beléptetése a Laravel API-n keresztül.

Fő metódus: OnLoginClicked

Lépései:

- kiolvassa az email és jelszó mezőket,
- kliens oldalon ellenőrzi, hogy ne legyenek üresek,
- JSON body-t készít email és password mezőkkel,
- POST kérést küld a /api/admin/login végpontra,
- siker esetén kinyeri a tokent,
- elmenti a tokent Preferences alá auth_token kulccsal,
- a felhasználót átnavigálja a MainPage oldalra.

Ez a metódus tehát a teljes admin session belépési kapu a MAUI oldalon.

23.2 MainPage

Ez az admin dashboard jellegű főoldal. A konstruktorban rögtön több betöltő metódus fut:

- LoadProducts()
- LoadUsers()
- LoadBids()
- LoadPayments()

Ezek párhuzamos üzleti nézeteket töltenek be ugyanarra az admin felületre.

Fontosabb metódusok:

- LoadProducts(): terméklista lekérése publikus API-ról
- LoadUsers(): admin felhasználólista lekérése Bearer tokennel
- LoadBids(): összes licit lekérése
- LoadPayments(): fizetések lekérése admin jogosultsággal
- LoadUserBids(int userId): egy user licitjeinek lekérése
- LogOutAdmin(...): tokenes kijelentkezés, majd token törlése lokális tárolóból
- OnBanClicked(...): user tiltása
- OnUnbanClicked(...): user tiltásának feloldása
- ShowProducts/ShowUsers/ShowBids/ShowPayments: UI szekciók láthatóságának kapcsolása

23.3 NewProduct

Feladata új termék feltöltése.

Fő metódusok:

- OnPickImageClicked(...): rendszer fájlválasztójából képet kér be
- OnSubmitProductClicked(...): összegyűjti a form mezőit és meghívja az ApiService.CreateProductAsync(...) metódust

Ez az oldal nem közvetlenül HttpClient-tel dolgozik, hanem az ApiService-en keresztül.

23.4 EditProductPage

Feladata meglévő termék lekérése, megjelenítése és frissítése.

Fontos metódusok:

- LoadProduct(): lekéri a terméket ID alapján és feltölti a form mezőit
- OnAppearing(): oldal megnyitásakor automatikusan betölti a terméket
- OnPickImageClicked(...): új képfájl kiválasztása
- OnUpdateProductClicked(...): csak a módosult mezőkből állít össze patch payloadot
- ResolveImageUri(...): a Laravelből érkező relatív image_url mezőből megpróbál teljes URL-t képezni

Az OnUpdateProductClicked(...) különösen fontos, mert nem teljes cserét akar küldeni, hanem a módosult mezőket gyűjti össze. Ez csökkenti a felesleges terhelést, és jobban illeszkedik a PATCH hiváshoz.

24. ApiService részletesen

Az ApiService a MAUI kliens központi API rétege a termék CRUD műveletekhez.

24.1 Konstruktor

Az ApiService() konstruktor létrehoz egy HttpClient példányt és beállítja a BaseAddress értékét http://localhost:8000/api/ címre.

Ez azt jelenti, hogy a service minden relatív hívást ehhez az alap URL-hez képest küld.

24.2 ApplyAuthHeader()

Feladata:

- kiolvassa a mentett auth_token értéket a Preferences tárból,
- törli az előző Authorization headert,
- ha van token, beállítja a Bearer <token> Authorization headert.

Ez a metódus minden védett API hívás előtt fontos, mert nélküle a Laravel auth:sanctum middleware 401 választ adhat.

24.3 CreateProductAsync(...)

Ez a metódus új terméket hoz létre multipart kérésen keresztül.

Mit csinál pontosan:

- auth header beállítás,
- MultipartFormDataContent létrehozás,
- string mezők hozzáadása a Laravel által várt mezőnevekkel,
- opcionális képfájl csatolása image_url néven,
- POST /api/products küldése,
- hiba esetén HTTP státusz és response body visszaadása.

Miért multipart: a képfeltöltés miatt a JSON body nem lenne elég.

24.4 GetProductAsync(int id)

Feladata egy termék lekérése szerkesztéshez.

Mit kezel le:

- auth header alkalmazása,
- GET /api/products/{id} hívás,
- a JSON válasz beolvasása,
- kétféle payload formátum támogatása:
    - ha a backend { "product": { ... } } alakban ad választ,
    - ha maga a gyökérobjektum a termék.

Ez a rugalmasság azért hasznos, mert a kliens kevésbé törik el kisebb response-shape változásoknál.

24.5 PatchProductAsync(...)

Ez a legösszetettebb metódus a service-ben.

Feladata:

- részleges termékfrissítés küldése,
- text-only update esetén PATCH + JSON payload használata,
- képet is tartalmazó frissítésnél multipart payload küldése,
- ha a Laravel backend nem fogadja el a közvetlen multipart PATCH-et, fallbackként POST + _method=PATCH módszerre vált.

Ez kifejezetten Laravel-kompatibilitási megoldás. Sok PHP backend a multipart PUT/PATCH kéréseket nehezebben kezeli, ezért a method spoofing bevett kompromisszum.

24.6 BuildProductMultipartContent(...)

Segédmetódus a multipart body összeállítására.

Feladata:

- mezők bepakolása a kérésbe,
- opcionális kép csatolása,
- szükség esetén _method mező hozzáadása.

Ez a metódus csökkenti a duplikációt a frissítési ágak között.

24.7 DeleteProductAsync(int id)

Egyszerű törlés:

- auth header beállítás,
- DELETE /api/products/{id} hívás,
- bool sikerességi érték visszaadása.

25. MAUI és Laravel API

Az alábbiak a jelenlegi fontos összekötések:

Auth

- MAUI: LoginPage.OnLoginClicked(...)
- Laravel API: POST /api/admin/login
- Backend controller: Api\AdminAuthController@login
- Válasz: token, token type, user objektum

Logout

- MAUI: MainPage.LogOutAdmin(...)
- Laravel API: POST /api/admin/logout
- Backend controller: Api\AdminAuthController@logout
- Követelmény: Bearer token

Product list

- MAUI: MainPage.LoadProducts()
- Laravel API: GET /api/products
- Backend controller: Api\ProductController@index
- Válasz: { "products": [...] }

Single product

- MAUI: ApiService.GetProductAsync(id)
- Laravel API: GET /api/products/{product}
- Backend controller: Api\ProductController@show
- Válasz: { "product": { ... } }

Create product

- MAUI: NewProduct.OnSubmitProductClicked(...) -> ApiService.CreateProductAsync(...)
- Laravel API: POST /api/products

- Backend controller: Api\ProductController@store
- Követelmény: Bearer token + admin jog + multipart body

Update product

- MAUI: EditProductPage.OnUpdateProductClicked(...) -> ApiService.PatchProductAsync(...)
- Laravel API: PATCH /api/products/{product}
- Fallback: POST /api/products/{product} _method=PATCH
- Backend controller: Api\ProductController@update

Delete product

- MAUI: törlési művelethez az ApiService.DeleteProductAsync(id) használható
- Laravel API: DELETE /api/products/{product}
- Backend controller: Api\ProductController@destroy

User list

- MAUI: MainPage.LoadUsers()
- Laravel API: GET /api/admin/users
- Backend controller: Api\UserController@index
- Követelmény: Bearer token + viewAny policy

Ban / unban

- MAUI: OnBanClicked(...), OnUnbanClicked(...)
- Laravel API: POST /api/admin/users/ban/{user} és POST /api/admin/users/unban/{user}
- Backend controller: Api\UserController@banUser, Api\UserController@unbanUser

Bids

- MAUI: MainPage.LoadBids()
- Laravel API: GET /api/bids
- Backend controller: Api\BidController@index

User bids

- MAUI: MainPage.LoadUserBids(userId)
- Laravel API: GET /api/admin/users/{user}/bids
- Backend controller: Api\BidController@userBids

Payments

- MAUI: MainPage.LoadPayments()
- Laravel API: GET /api/payments
- Backend controller: Api\PaymentController@index
- Válasz: { "payments": [...] }
- Követelmény: Bearer token + admin policy

26. MAUI oldali modellek és json válaszok

Models/Product

Fő mezők:

- Id
- Name
- Category
- Description
- ExtendedDescription
- ImageUrl
- StarterBid
- BidStartDate
- BidEndDate

ProductResponse

Azért kell külön wrapper, mert a backend nem nyers tömböt küld, hanem:

```
json
{
	"products": [ ... ]
}
```

PaymentResponse

Ugyanez a minta fizetéseknél:

```
json
{
	"payments": [ ... ]
}
```

Models/User, Models/Bid, Models/Payment

Ezek a listanézetekhez szükséges admin adatstruktúrákat reprezentálják.

27. Auth és tokenkezelés a MAUI appban

A tokenkezelés jelenlegi folyamata:

1. login után a backend Sanctum tokent ad vissza,
2. a MAUI kliens ezt Preferences.Set("auth_token", token) formában eltárolja,
3. a védett hívások előtt a kliens beállítja az Authorization: Bearer <token> fejlécet,
4. logoutkor a kliens szerveroldalon is visszavonja az aktuális tokent,
5. végül lokálisan törli az auth_token kulcsot.

28. Gyakori MAUI hibák és fejlesztői megjegyzések

28.1 Tokenes végpont 401-et ad

Ok lehet:
- nincs elmentve token,
- nem lett beállítva Authorization header,
- lejárt vagy törölt token,
- nem admin userrel történt login.

28.2 Képfeltöltés sikertelen

Ok lehet:
- rossz field name,
- fájl nem létezik a kiválasztott pathon,
- backend validáció elutasítja a feltöltést,
- a Laravel storage/public konfiguráció nincs rendben.

29. Rövid belépési út új fejlesztőnek

Ha most csatlakozol a projekthez, a minimum lépéssor:

```
composer install
copy .env.example .env
php artisan key:generate
php artisan migrate --seed
php artisan storage:link
npm install
composer dev
```

Külön consolon: php artisan schedule:work

Ezután:

- jelentkezz be az admin@example.com felhasználóval,
- nézd át a routes/web.php fájlt,
- nézd át a routes/api.php fájlt,
- ellenőrizd a ProductController, BidController és PaymentController működését,
- indítsd el a licitAdminDashboard MAUI appot Windows targeten,
- teszteld le a login, termék létrehozás és user lista API-hívásokat,